

Class vs Struct in C#:

- **Type:**
 - Class → Reference type (stored on heap).
 - Struct → Value type (stored on stack).
- **Default Constructor:**
 - Class → Can define default (parameterless) constructors.
 - Struct → Cannot define a default constructor (compiler provides one).
- **Inheritance:**
 - Class → Supports inheritance (except multiple inheritance).
 - Struct → Cannot inherit from another struct or class (but can implement interfaces).
- **Nullability:**
 - Class → Can be null.
 - Struct → Cannot be null unless declared `Nullable<T>` or `T?`.
- **Memory Allocation:**
 - Class → Allocated on heap, garbage collected.
 - Struct → Allocated on stack (usually), lightweight.

We Use class for complex objects with behavior and inheritance, use struct for small, immutable, lightweight data.

Other Class Relationships

1. Inheritance (IS-A Relationship)

- **Definition:** A derived class extends a base class, inheriting fields, properties, and methods.
- **Nature:** Strong relationship; expresses hierarchy and reuse.
- **Key Idea:** “Child is a Parent.”
- **Polymorphism Support:** Yes, through virtual, override, and abstract.

2. Association (KNOWS-A Relationship)

- **Definition:** A broad relationship where one class references another.
- **Nature:** Looser than inheritance; can be one-to-one, one-to-many, or many-to-many.

- **Key Idea:** “Class A **knows about** Class B.”
- **Lifetime Dependency:** Independent — neither class necessarily controls the other’s lifecycle.

3. Aggregation (HAS-A Relationship, Weak Ownership)

- **Definition:** A specialized form of association where one class contains or groups another, but the part can exist independently.
- **Nature:** Represents a “whole–part” relationship with shared lifetime.
- **Key Idea:** “Class A **has a** Class B, but Class B can live without Class A.”
- **Lifetime Dependency:** Independent — destroying the container does not destroy the part.

4. Composition (HAS-A Relationship, Strong Ownership)

- **Definition:** A stronger form of aggregation where one class owns another, and the lifetime of the part depends on the whole.
- **Nature:** Tight coupling; part cannot exist without the whole.
- **Key Idea:** “Class A **owns a** Class B, and if A dies, B dies too.”
- **Lifetime Dependency:** Dependent — destroying the container destroys the contained.

5. Dependency (USES-A Relationship)

- **Definition:** A temporary relationship where one class depends on another for execution of a method or operation.
- **Nature:** The weakest relationship; typically seen when a method in one class accepts another class as a parameter, local variable, or return type.
- **Key Idea:** “Class A **uses** Class B.”
- **Lifetime Dependency:** Very short-lived; only during execution.

6. Realization (Interface Implementation)

- **Definition:** A class promises to provide implementations for an interface’s methods and properties.
- **Nature:** A contract between abstraction (interface) and implementation (class).
- **Key Idea:** “Class A **realizes** Interface B.”
- **Polymorphism Support:** Yes, since multiple classes can realize the same interface.

Quick Summary:

- **Inheritance** → **IS-A** (strong hierarchy).
- **Association** → **KNOWS-A** (general relationship).

- **Aggregation** → **HAS-A (weak ownership)** (independent lifetime).
- **Composition** → **HAS-A (strong ownership)** (dependent lifetime).
- **Dependency** → **USES-A** (temporary use).
- **Realization** → **CONTRACT** (implements interface).